# FlyBye

## Release-Readiness Audit

Mobile (Flutter) & Web (React / Lovable) applications

| | |
|---|---|
| **Prepared for** | ahmed@bolderapps.com |
| **Scope** | flybye (mobile) + flybywebapp (web) |
| **Date** | July 14, 2026 |
| **Classification** | Internal — engineering handoff |

# Contents

**1. Executive Summary** 3

**2. Architecture & Backend Findings** 3

2.1 Authentication is not unified . . . 3

2.2 Databases are not the same datasource . . . 3

2.3 Cross-cutting blockers . . . 4

**3. Mobile App (flybye) — Findings** 5

3.1 Blockers . . . 5

3.2 Major . . . 5

3.3 Minor . . . 6

**4. Web App (flybywebapp) — Findings** 7

4.1 Blockers . . . 7

4.2 Major . . . 7

4.3 Minor . . . 7

**5. Consolidation Path (releasing both together)** 9

5.1 Suggested attack order . . . 9

5.2 Severity legend . . . 9

# 1. Executive Summary

**VERDICT — FlyBye is not one product with two clients. It is two independently built apps that reimplement the same idea on two separate, mutually incompatible Supabase backends. They do not share accounts or data, and neither is currently shippable without the work below.**

This audit covers two codebases delivered together: **flybye** (a Flutter mobile app) and **flybywebapp** (a Vite / React web app generated with Lovable and wrapped with Capacitor). Both implement the same concept — "meet verified travelers at your airport" — but they were built as separate products with different data models, on different Supabase projects.

The core user loops are, encouragingly, **genuinely wired to Supabase** in both apps — this is not a mock-data prototype. However, release-critical features are stubbed or broken in each, and a deeper architectural problem blocks the stated goal of launching mobile and web **together**: the two backends cannot be merged without a deliberate schema migration.

### Headline numbers

| Area | Mobile (flybye) | Web (flybywebapp) |
|---|---|---|
| Backend Supabase project | `rxipszdcbvrevuyqpncq` (docs) | `nqgpciewfajrcingdlgl` |
| Runnable out of the box? | No — no `.env` committed | Yes |
| Core loop wired to real DB? | Yes | Yes |
| Release blockers found | **4** | **2** |
| Major issues | **3** | **3** |
| Store-shippable today? | No | No (Capacitor build is a hosted URL) |

# 2. Architecture & Backend Findings

The most important finding is structural. The two apps point at **two different Supabase projects**, which means two entirely separate authentication user pools and two separate databases. Even where table names coincide, the schemas are incompatible.

## 2.1 Authentication is not unified

Both apps use Supabase Auth (GoTrue) email/password — the *mechanism* is identical — but because they target different projects, **an account created on the web cannot sign in on mobile**, and vice versa. The web app additionally layers a Lovable OAuth wrapper (Google / Apple / Microsoft) that mobile does not have. Password-reset redirects also differ by design (`flybye://reset-password` deep link vs. web origin URL).

## 2.2 Databases are not the same datasource

Beyond being separate projects, the schemas diverge in ways that prevent a naive merge. Running both migration sets against one project would cause shared objects (`profiles`, `matches`, `messages`, `handle_new_user`) to collide — the last one applied silently breaks the other app.

| Concept | Mobile schema | Web schema |
|---|---|---|
| **profiles PK** | `id = auth.users.id` (1-to-1) | `id` is a separate UUID; `user_id` → auth user |
| **new-user trigger** | inserts (id, display_name, birthdate) | inserts (user_id, display_name) |
| **Matching** | swipes table + mutual-like trigger creates match | request/accept: requester_id, target_id, status |
| **messages** | column `body`, realtime stream | column `content`, 3-second polling |
| **flights** | one active airport presence / user | multiple full itinerary rows |
| **Verification** | verification_requests table + verifications bucket | profiles.verification_status + id-documents bucket + missing Persona fn |
| **Subscriptions** | subscriptions table + RevenueCat webhook | none |

## 2.3 Cross-cutting blockers

**A1** BLOCKER **Two separate Supabase projects — no shared auth or data**

Web signup cannot log in on mobile; there is no cross-platform account or data parity. Unifying requires choosing one project and migrating.

**A2** BLOCKER **Incompatible schemas on shared table names**

profiles, matches, messages and the `handle_new_user` trigger differ incompatibly. Both apps cannot point at one database without a deliberate migration and compatibility layer.

**A3** BLOCKER **Flutter app has no committed backend configuration**

`main.dart` reads SUPABASE_URL / SUPABASE_ANON_KEY from a `.env` declared as an asset but absent from the repo. The app currently boots to a “Boot error” screen and is not runnable until the file is created.

→ flybye/lib/main.dart, flybye/.env (missing)

**A4** BLOCKER **Web “mobile” build loads a hosted Lovable preview URL**

capacitor.config.ts sets `server.url` to a `lovableproject.com` preview address. A native build just loads Lovable’s hosted web preview rather than bundled code — not shippable to an app store as-is.

→ flybywebapp/capacitor.config.ts

# 3. Mobile App (flybye) — Findings

Flutter + Riverpod + go_router on Supabase. The core loop (auth → onboarding → flight → swipe → match → chat) is genuinely functional. Report/block, discover RPC, realtime chat, and flight lookup are real. The items below are the gaps.

## 3.1 Blockers

**M1** BLOCKER **Account deletion is fake**

Settings → “Delete account” shows a confirm dialog then only displays an “email support@flybye.app” snackbar — nothing is deleted. Apple Guideline 5.1.1(v) requires functional in-app deletion for any app with account creation.

→ flybye/lib/features/profile/screens/settings_screen.dart

**M2** BLOCKER **Password reset / change is unreachable**

`listenForPasswordRecovery()` is defined but never called anywhere. When the user taps the reset email link, the SDK creates a recovery session and the router sends the now-“logged-in” user to /discover — the New Password screen is never shown. Settings → Change Password routes to the same dead flow.

→ flybye/lib/features/auth/services/auth_service.dart

**M3** BLOCKER **No backend .env (see A3)**

The app cannot run until Supabase credentials are provided. Blocking for any build or test.

**M4** BLOCKER **Paywall is a “coming soon” placeholder**

RevenueCat is removed; the paywall renders static text; `currentTier()` hardcodes `standby` and all tier-gating code is dead. This is intentional for a free v1.0, but the live “Upgrade” button leads to a dead screen — a review risk if left visible.

→ flybye/lib/features/subscription/screens/paywall_screen.dart

## 3.2 Major

**M5** MAJOR **No profile editing after onboarding**

`updateBio()` / `updateVibes()` exist but are never called; the profile screen is read-only and photos can only be set once during onboarding. Core expectation for a dating/social app is missing.

→ flybye/lib/features/profile/data/profile_repo.dart

**M6** MAJOR **Verification is submit-only**

Selfie capture and upload work, but there is no in-app status view, resubmit, or rejection feedback, and nothing client-side flips `is_verified`. “Verification pending” can display indefinitely.

→ flybye/lib/features/auth/screens/onboarding_screen.dart

**M7** **MAJOR** **Avatars use public URL only**

`avatarUrl()` returns `getPublicUrl()` and never signs. If the `avatars` bucket is private, all avatars fail to load. Verify bucket ACL.

→ flybye/lib/features/profile/data/profile_repo.dart

## 3.3 Minor

- Dark theme is deliberately disabled (in-app screens render invisible text in dark mode); pinned to light as a workaround.
- Sign-up “Terms & Privacy Policy” is plain, non-tappable text — a possible compliance nit.
- Help/FAQ and Privacy links open flybye.app/support and /privacy — confirm these URLs are live before launch.
- Discover swipe index is not reset on feed reload; mutual-match detection depends on a DB trigger having already run (minor race).

# 4. Web App (flybywebapp) — Findings

Vite + React + shadcn/ui + Supabase, Lovable-generated. Auth, discover/swipe data, match accept/decline, flights CRUD, profile text edit, ID upload to storage, location sharing, and the offline / PWA / sync-queue layer are all genuinely implemented. The critical problem is that verification — which gates the entire experience — is broken.

## 4.1 Blockers

**W1** BLOCKER **Verification is broken, and it gates the whole app**

The Persona kickoff invokes an edge function `start-verification` that **does not exist** (only airport-pois and process-email-queue are deployed). Worse, **no code path anywhere sets a user to "verified"** — the manual ID upload only sets status `processing`, and there is no admin UI, trigger, or migration to advance it. Yet Discover blocks all swipe / match / chat behind `verified`. Net effect: **no web user can ever swipe, match, or message.** The app is end-to-end unusable as shipped.

→ flybywebapp/src/components/VerificationStatusCard.tsx, src/pages/Discover.tsx, src/hooks/useVerificationStatus.ts

**W2** BLOCKER **Capacitor build loads a hosted Lovable URL (see A4)**

Not a real native app; unshippable as a Capacitor build until the app is bundled and served locally.

→ flybywebapp/capacitor.config.ts

## 4.2 Major

**W3** MAJOR **Avatar "upload" is a paste-a-URL text box**

The profile avatar control is an `<Input placeholder="Paste image URL">` writing to `avatar_url` — no file picker, no storage upload. The real upload pattern exists in IDUploadCard but was not reused here.

→ flybywebapp/src/pages/Profile.tsx

**W4** MAJOR **Airport map & POIs are dead — airports table is never seeded**

The map/POI view calls airport-pois, which returns 404 when the airport row is missing. The `airports` table is created but no migration or seed inserts any rows, so every airport shows an empty map.

→ flybywebapp/supabase/functions/airport-pois/index.ts, src/components/TerminalMap.tsx

**W5** MAJOR **Chat & matches are polling, not realtime**

Chat polls every 3s and location every 15s; the Matches list only fetches on mount, so incoming requests do not appear until manual reload. No Supabase realtime channels are used anywhere despite the "real-time" positioning.

→ flybywebapp/src/pages/Chat.tsx, src/pages/Matches.tsx

## 4.3 Minor

- Landing page stats ("50K+ Travelers", "98% Verified") and testimonials are fabricated mock data; some footer links use href="#" though real routes exist.
- "Nearby flyers" count is an approximation (max of two counts), not a distinct-user count.

- Social login depends on Lovable Cloud OAuth being configured in the backend, or the buttons error at runtime.
- No payments: only legal copy references subscriptions, plus a commented-out Stripe create-checkout function that does not exist.

# 5. Consolidation Path (releasing both together)

Because both apps must ship on one account system, a backend consolidation is the true prerequisite. Recommended sequence:

1. **Pick the canonical schema** — profiles PK, matching model, messages, flights, verification, subscriptions. The Flutter schema is the more store-ready and internally consistent base; the web app would be adapted onto it (or vice-versa). This is the pivotal decision.
2. **Consolidate to one Supabase project**; add compatibility views / RPCs so the non-canonical client keeps working during migration.
3. **Move clients off direct table writes** to RPCs where semantics differ (swipes / match requests, message send, verification transitions, account deletion).
4. **Regenerate Supabase types** for the web app after every schema change.
5. **Add integration tests** for RLS-sensitive flows before migrating any users.

The repository's own `TECHNICAL_DOCUMENTATION.md` ("Backend Consolidation Notes") sketches an accurate version of this path and is worth following.

## 5.1 Suggested attack order

| # | Step | Why first |
|---|---|---|
| 1 | **Decide the canonical product / schema** | Everything else depends on it |
| 2 | **Fix web verification (W1)** | The web app is unusable without it |
| 3 | **Consolidate backends onto one Supabase project** | Unblocks shared accounts (A1/A2) |
| 4 | **Mobile store blockers: deletion, reset, .env (M1–M3)** | Required for App Store |
| 5 | **Real avatar upload (both), seed airports (W4), paywall decision** | Core UX / data |
| 6 | **Realtime for web chat/matches; polish** | Quality & parity |

## 5.2 Severity legend

| Level | Meaning |
|---|---|
| **BLOCKER** | Prevents launch / core flow is broken or non-compliant |
| **MAJOR** | Significant missing/degraded feature; fix before public launch |
| **MINOR** | Polish, cosmetic, or config verification |

---

Prepared as an investigation-only deliverable. No source files were modified. Findings were verified by direct source inspection of both repositories, their Supabase migrations, and the project's technical documentation.